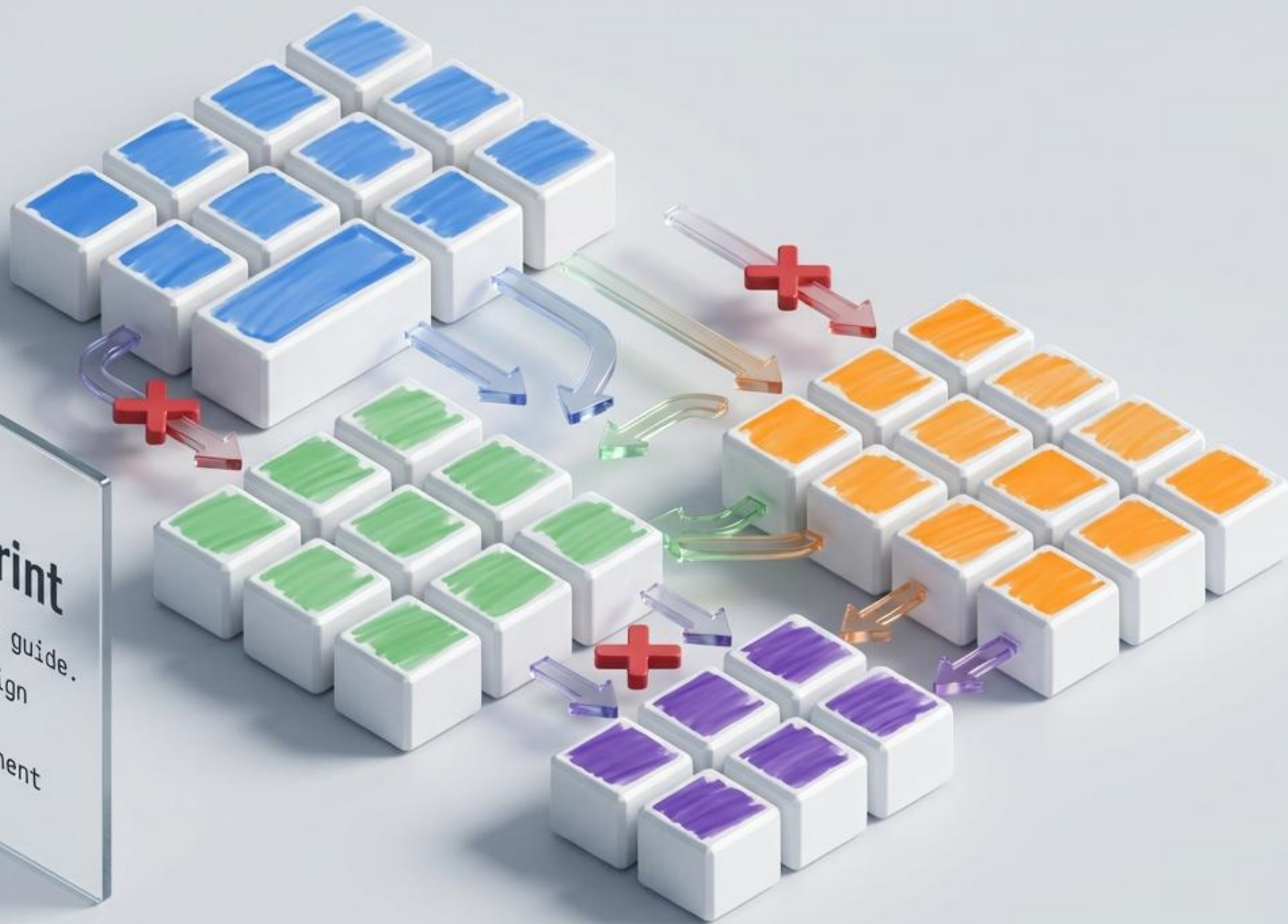


Component-Driven Architecture Blueprint

A scalable, rule-based structural guide.
Built on the J.P. Morgan Salt Design
System (@salt-ds).
Optimized for predictable component
extraction and feature scaling.



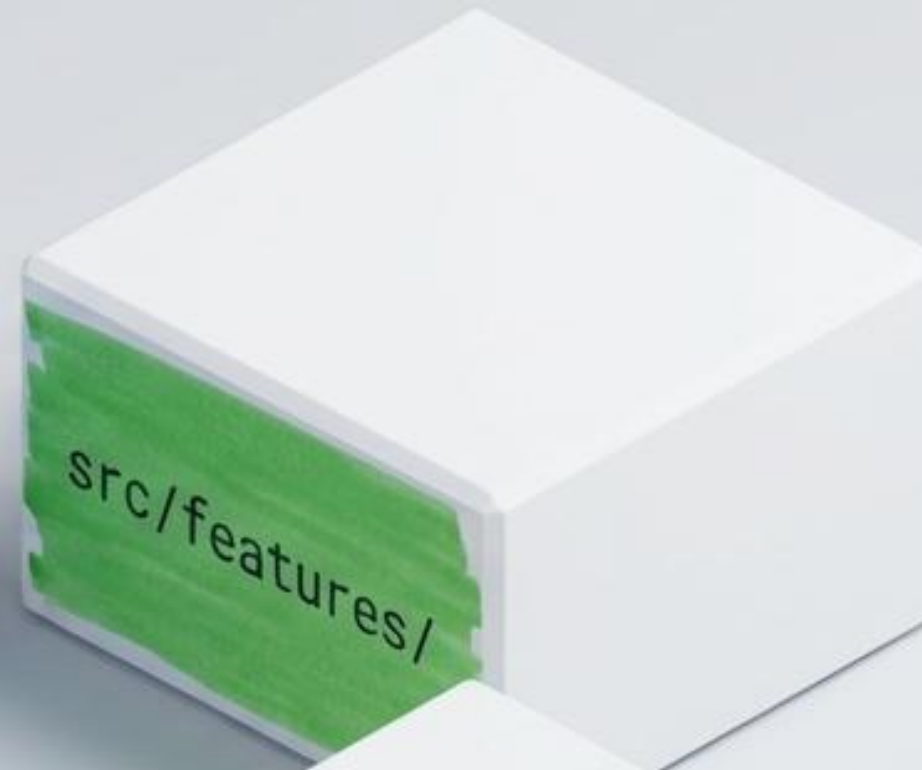
App Shell

Router-centric
orchestration.



Business Logic

Feature-based
vertical slices.



UI Components

Atomic design
primitives.

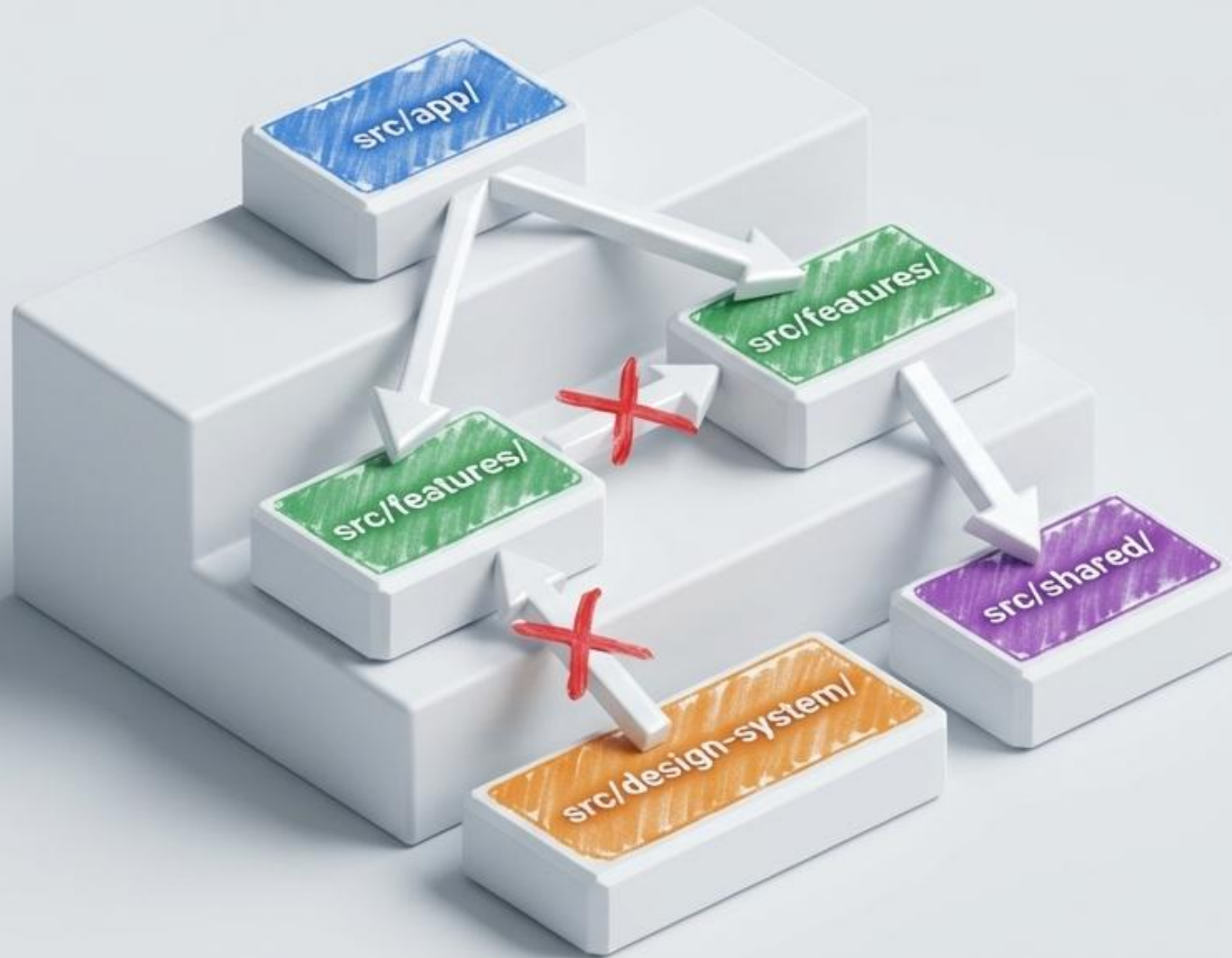


Shared Library

Domain-agnostic
utilities.



Structural Gravity Enforces Strict Dependency Flow



Rule 1: Dependencies only flow downward toward design-system and shared.

Downward dependency flow is strictly enforced, preventing circular references and maintaining a clear architectural hierarchy.

Rule 2: Features never import from other features.

Feature blocks must remain independent, communicating only through the app shell or shared libraries.

Rule 3: The design system remains entirely ignorant of business logic.

The design system is a foundational layer and must never depend on or be aware of feature-specific logic.

The Golden Rule: Only the design-system/ layer and main.tsx import from @salt-ds/* directly.

Direct Consumption



atoms/, molecules/,
organisms/, templates/

Strictly Isolated



features/components/,
features/pages/, app/routes/

Feature components compose wrapped design-system components, never raw Salt components.

Bedrock Packages Powering the Foundation



SaltProviderNext configures the application root in `main.tsx`.

Global CSS
(`@salt-ds/theme/index.css`) must be imported once to load all custom properties.

Requires Open Sans and PT Mono font packages.

Dynamic Scaling via the Token Engine

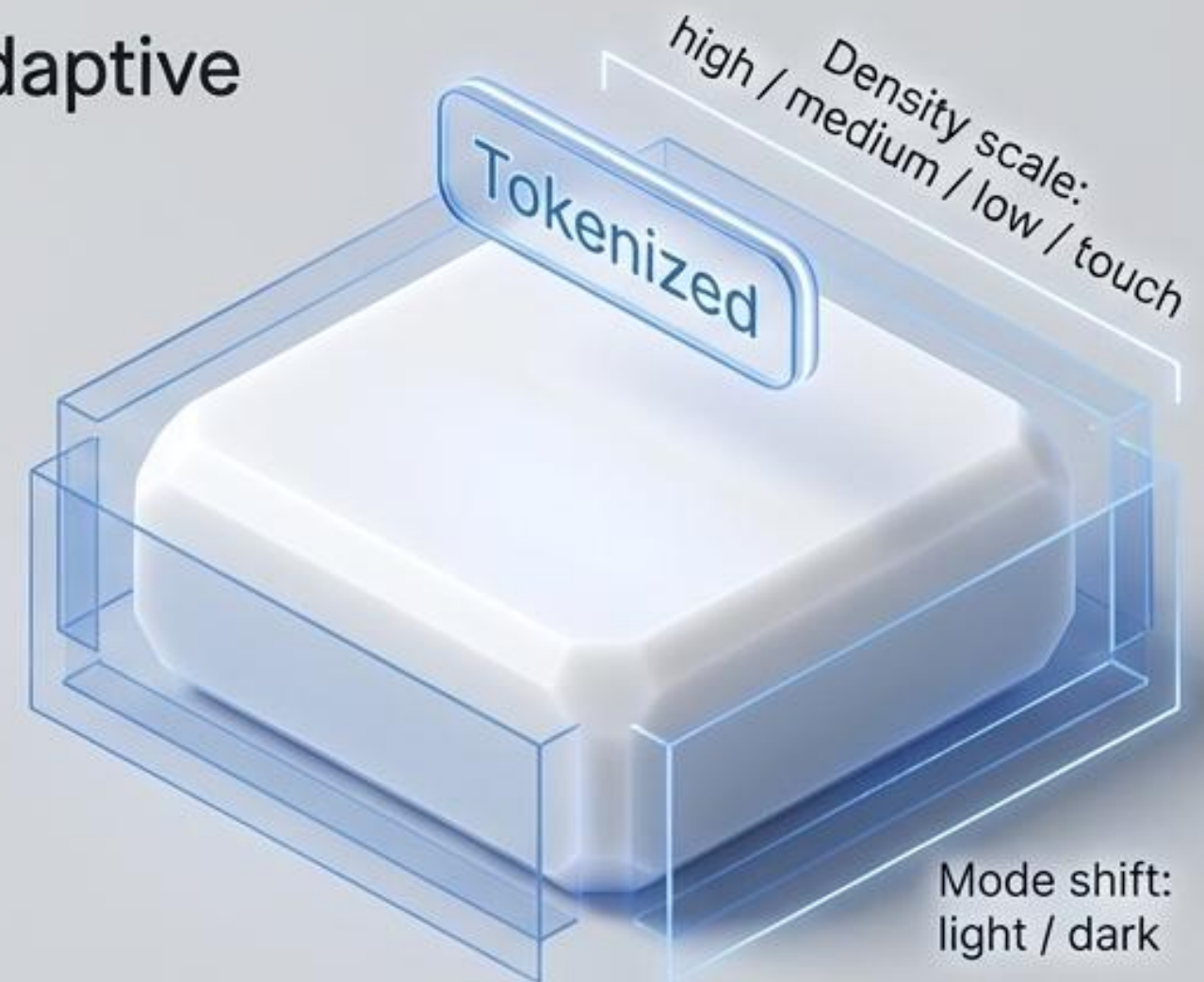
Fragile



Never use raw hex colors or static pixel values.

Distributes 1,844 design tokens via `--salt-*` CSS custom properties.

Adaptive



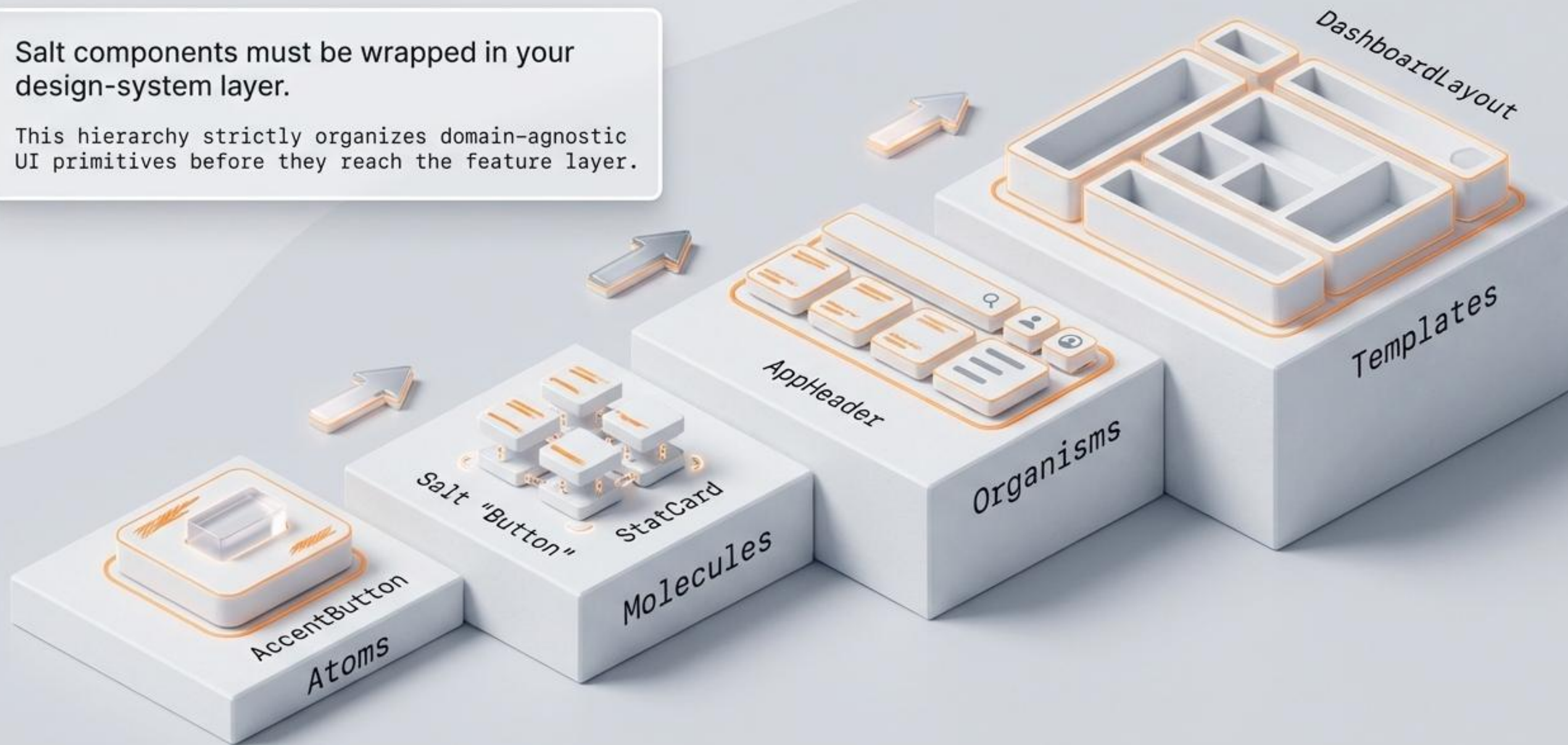
Spacing:
`--salt-spacing-100`
scales dynamically.

Colors:
`--salt-container-primary`
adapts to mode.

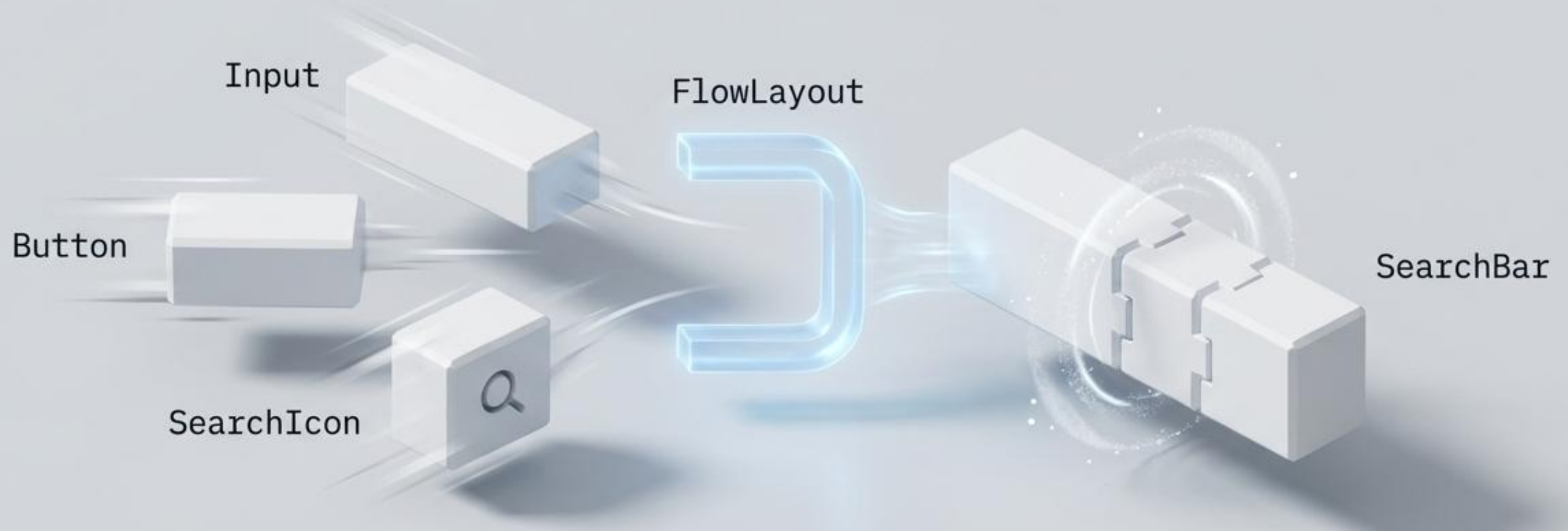
The Atomic Escalation Staircase

Salt components must be wrapped in your design-system layer.

This hierarchy strictly organizes domain-agnostic UI primitives before they reach the feature layer.



Assembling Atoms into Functional Molecules



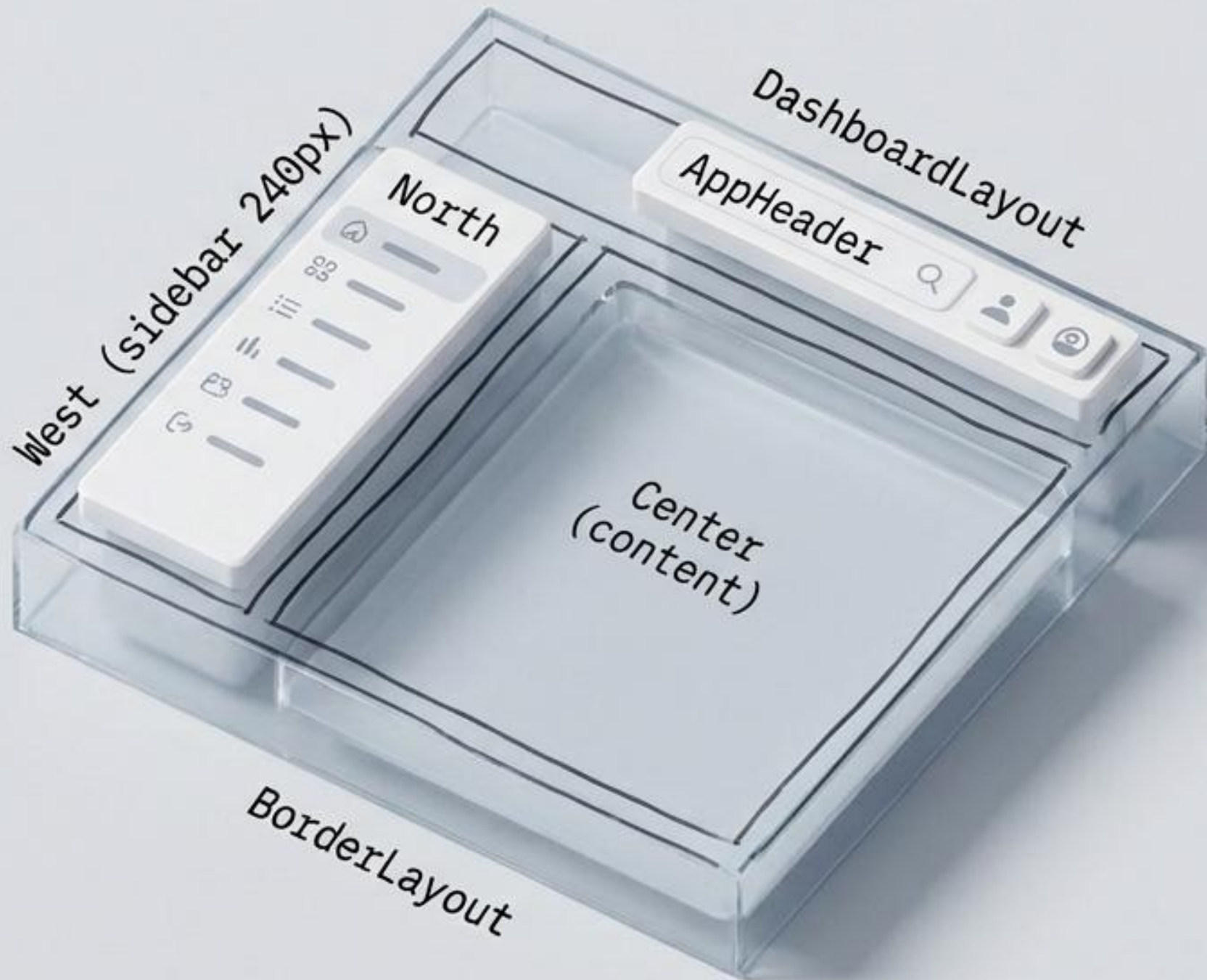
Atoms:

Wrap a single Salt component with preset props.
Example: AccentButton sets sentiment='accented'

Molecules:

Compose multiple Salt components for minor interaction logic.
Example: FormField molecule combines FormFieldLabel, Input, and StatusAdornment.

Macro-Scaffolding with Salt Layout Primitives



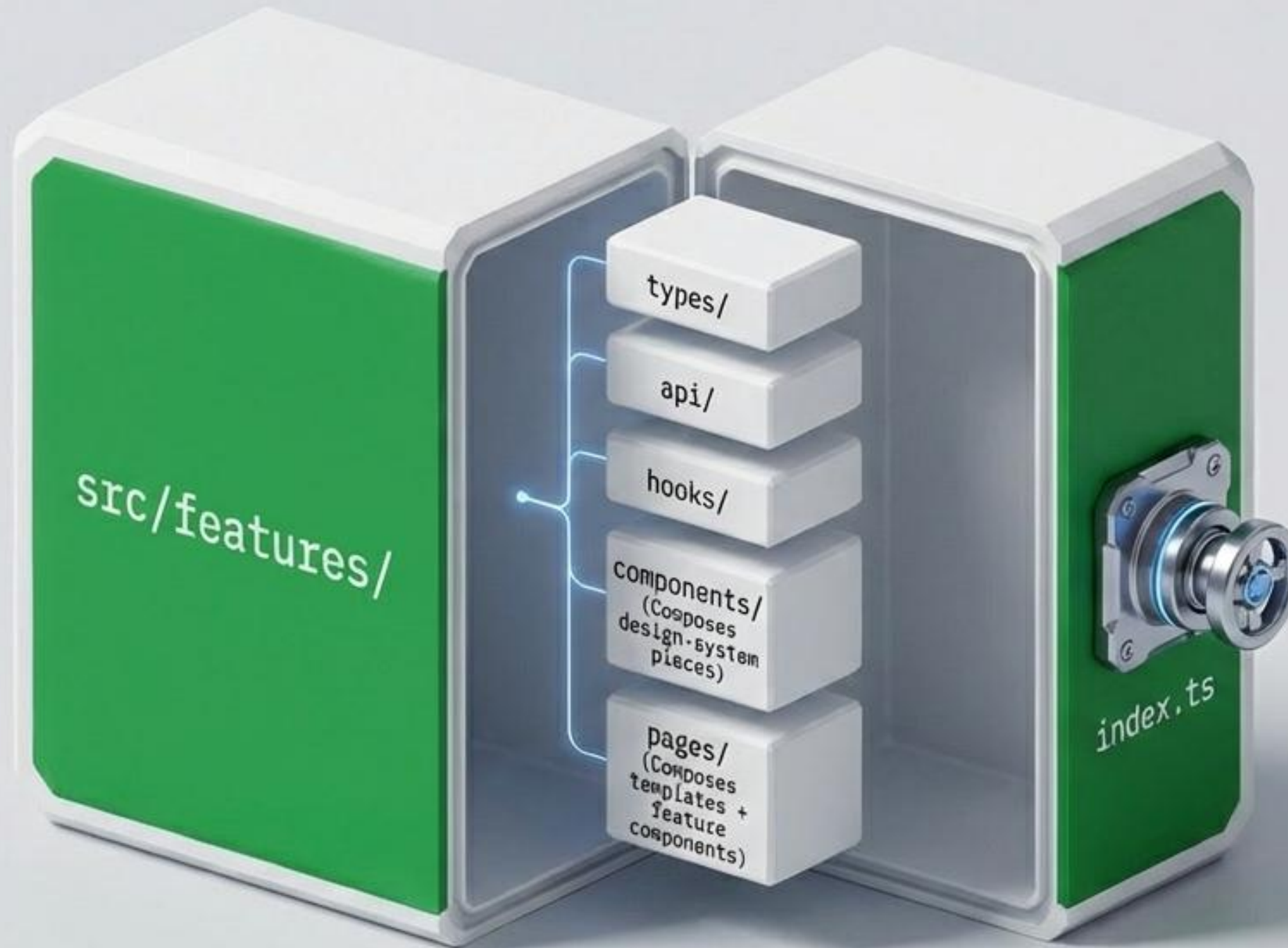
Eliminate raw CSS flexbox and grid.

Utilize Salt layout primitives:

- BorderLayout
- FlowLayout
- StackLayout
- SplitLayout
- GridLayout

Organisms build complex sections (e.g., VerticalNav), while Templates provide empty slots for layout strategy.

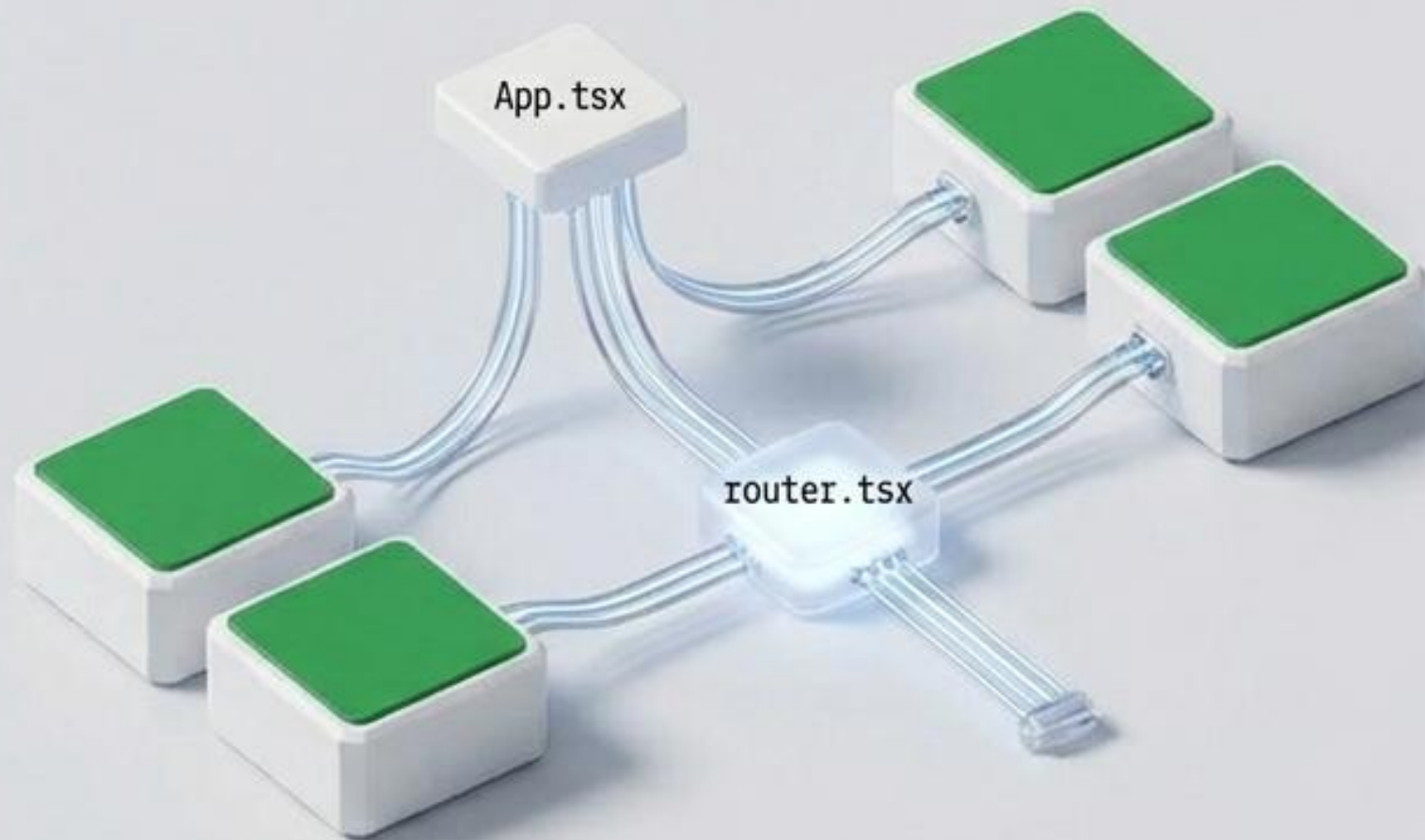
Deconstructing Feature-Based Vertical Slices



- Each feature is entirely self-contained with its own state and domain data.
- Cross-feature communication happens exclusively through the shared/ library.
- The `index.ts` barrel export defines the strict public API for the feature.

The Orchestrators: App Shell & Shared Library

src/app/



App Shell: The thin orchestration layer. Declares routes via `createBrowserRouter` and composes global providers. Zero business logic.

src/shared/



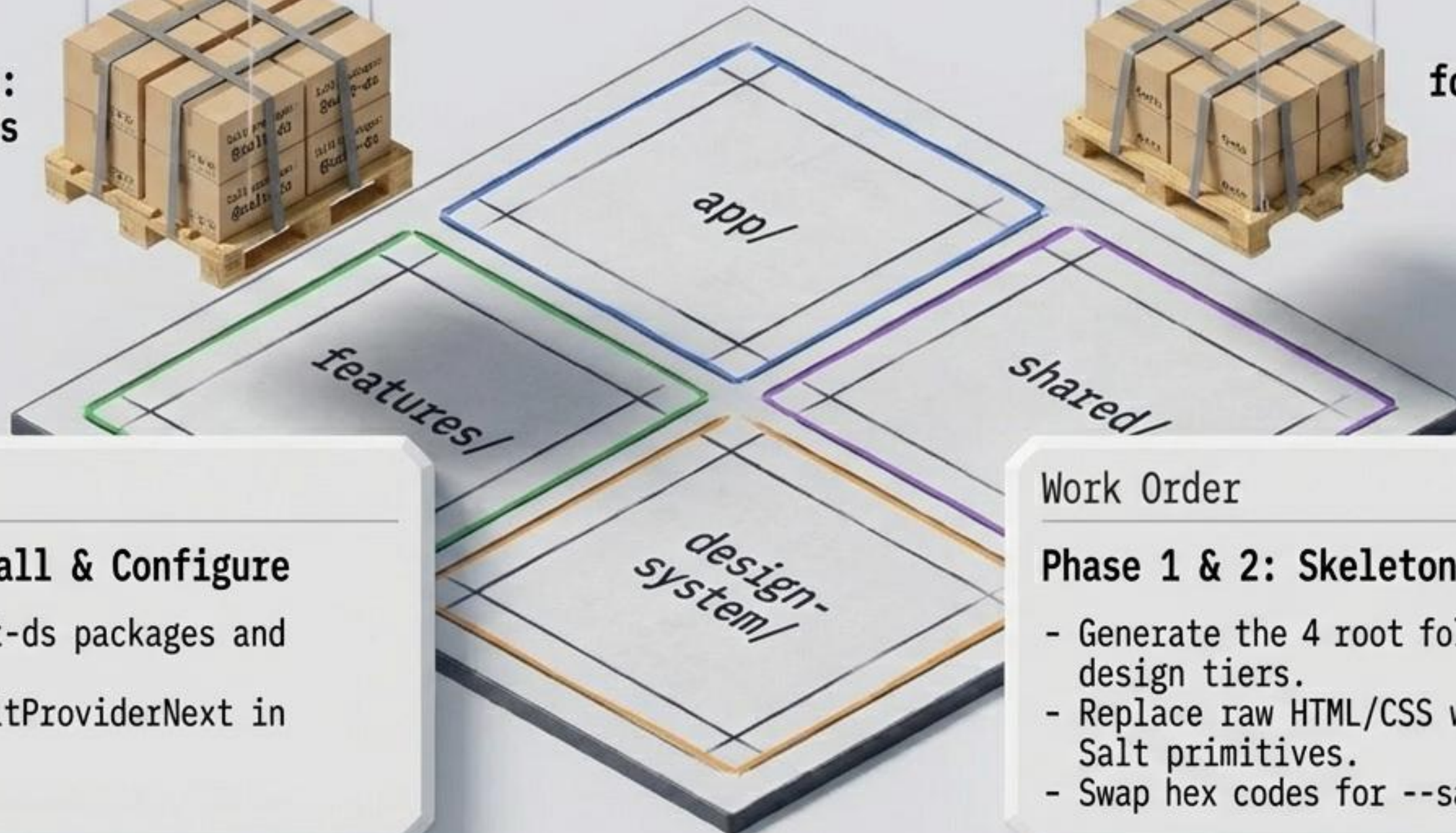
Shared Library: Domain-agnostic utilities and hooks required by two or more features.

Assembly Phase 1 & 2: Structural Foundation

Salt packages:
@salt-ds



fonts



Work Order

Phase 0: Install & Configure

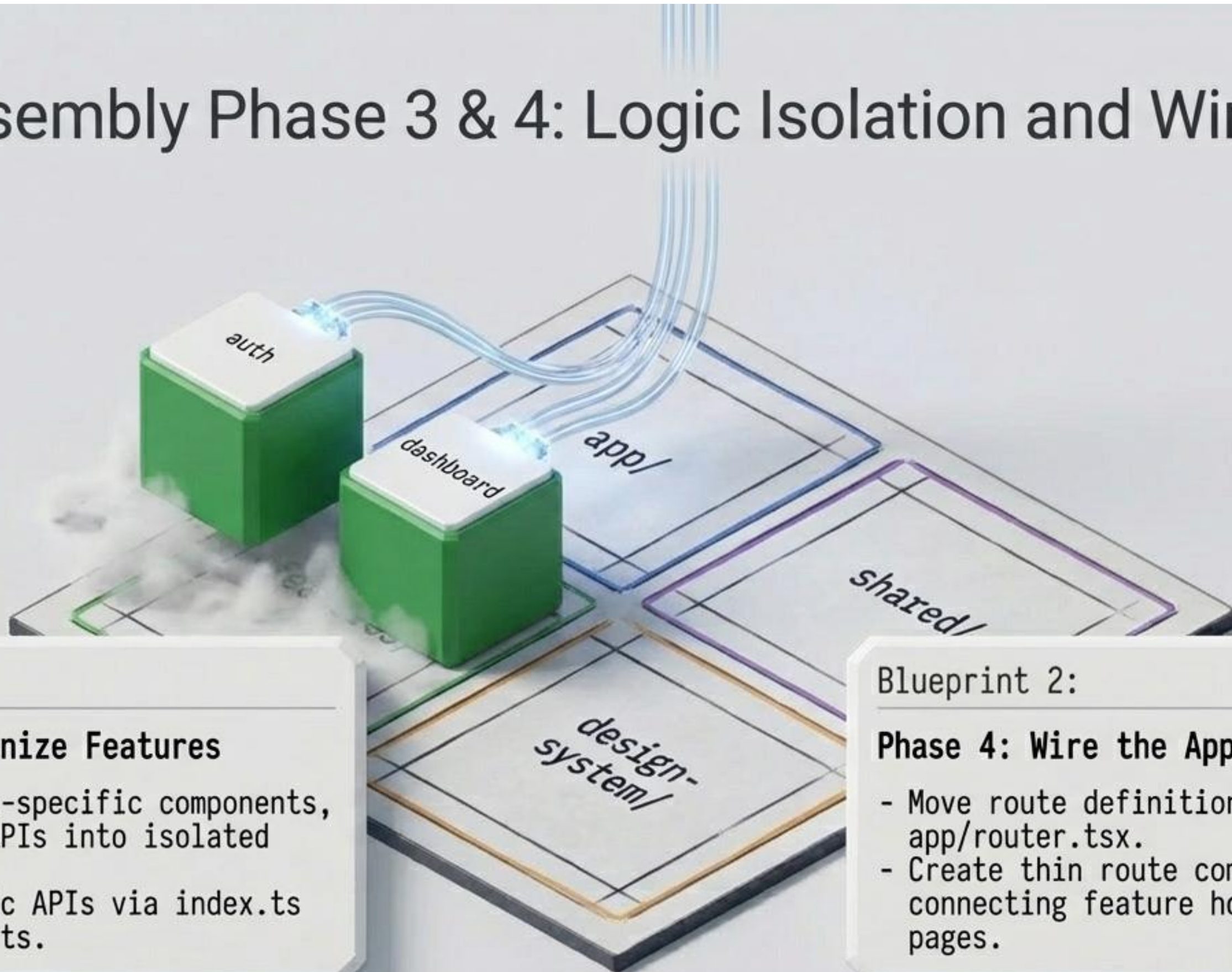
- Install @salt-ds packages and fonts.
- Configure SaltProviderNext in main.tsx.

Work Order

Phase 1 & 2: Skeleton & Extraction

- Generate the 4 root folders and atomic design tiers.
- Replace raw HTML/CSS with wrapped Salt primitives.
- Swap hex codes for --salt-* properties.

Assembly Phase 3 & 4: Logic Isolation and Wiring



Blueprint 1:

Phase 3: Organize Features

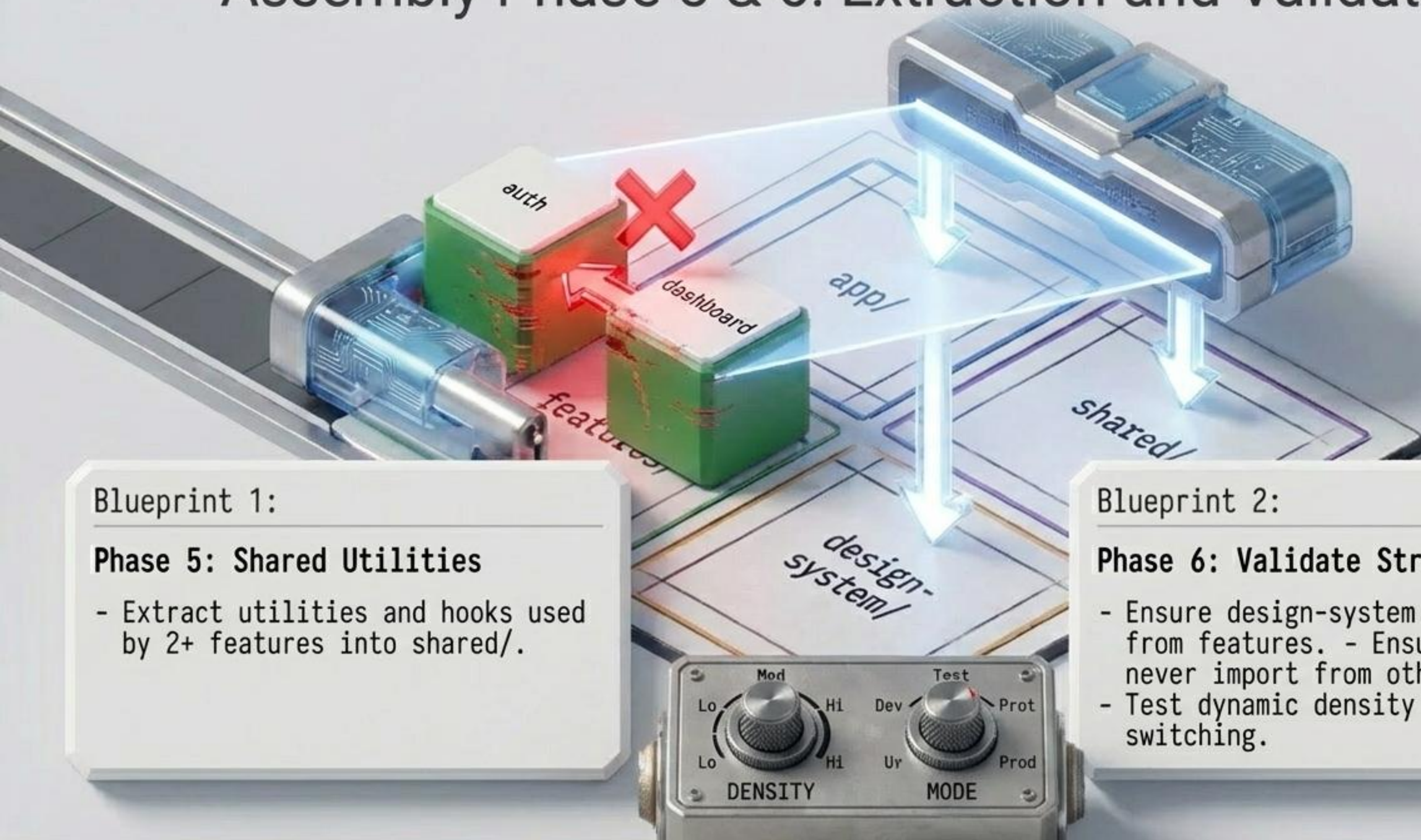
- Group domain-specific components, hooks, and APIs into isolated folders.
- Expose public APIs via `index.ts` barrel exports.

Blueprint 2:

Phase 4: Wire the App Shell

- Move route definitions to `app/router.tsx`.
- Create thin route components connecting feature hooks to feature pages.

Assembly Phase 5 & 6: Extraction and Validation



Blueprint 1:

Phase 5: Shared Utilities

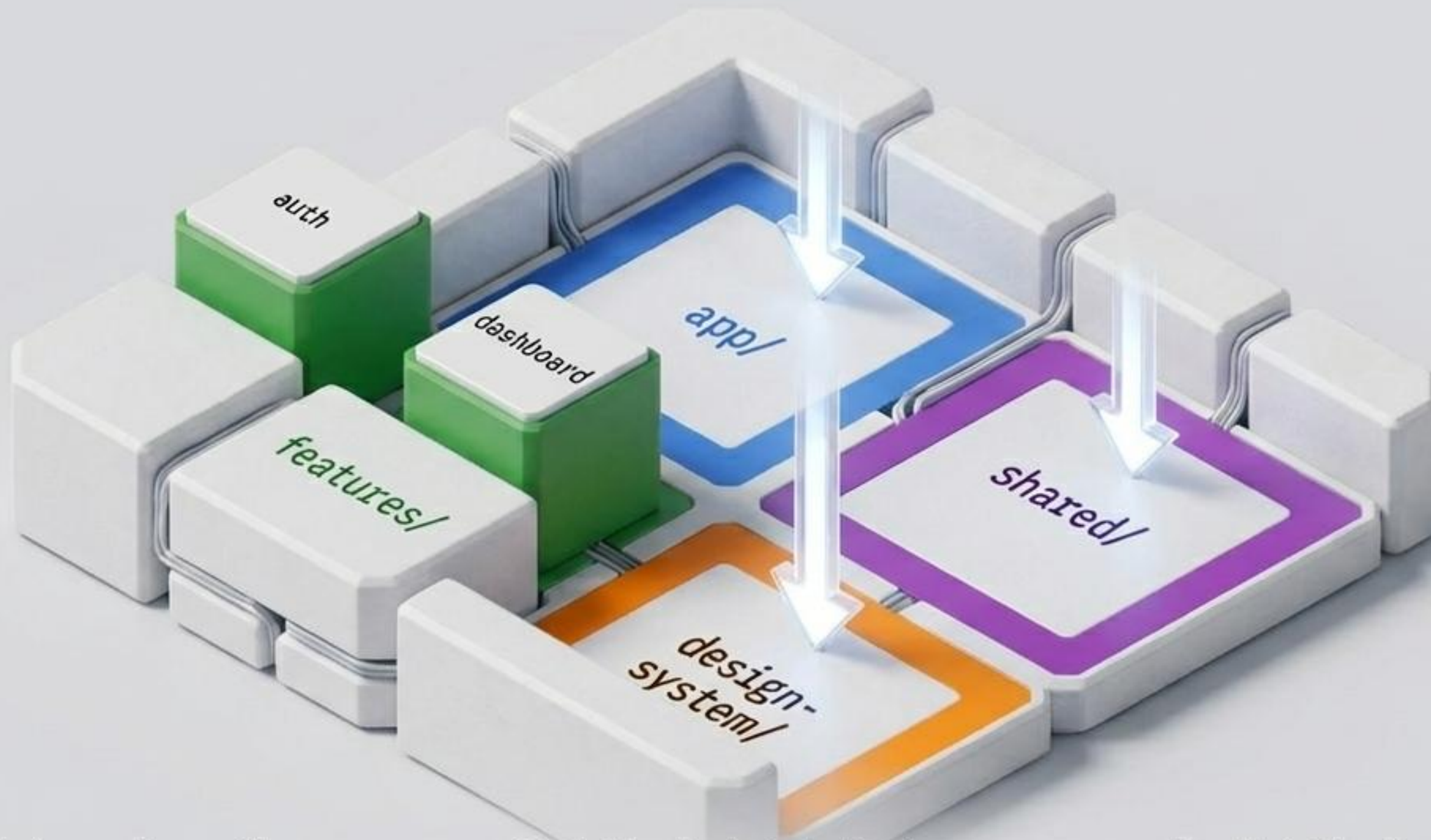
- Extract utilities and hooks used by 2+ features into shared/.

Blueprint 2:

Phase 6: Validate Strict Flow

- Ensure design-system never imports from features.
- Ensure features never import from other features.
- Test dynamic density and mode token switching.

The Scalable Assembly Blueprint Maintained



Component-Driven: Every UI element is built on wrapped, tokenized Salt primitives.



Strictly Isolated: Business domains remain entirely decoupled from one another.



Predictably Scalable: Complex interfaces emerge naturally from simple, atomic assembly rules.